

ReactJS for Frontend Development

- Introduction to ReactJS
- Understanding JSX
- Components
- State and Props
- Event Handling
- Conditional Rendering
- Lists and Forms
- React Hooks

Introduction to ReactJS

- ReactJS is a JavaScript library for building user interfaces.
- Developed by Facebook(Meta).
- Used for creating single-page applications.
- Uses component-based architecture.

Features of ReactJS

- Component-Based
- Virtual DOM
- Reusable Code
- Fast Rendering
- Easy to Learn
- One-way Data Binding

What is Virtual DOM?

- Virtual DOM is a lightweight copy of the real DOM.
- React updates Virtual DOM first.
- Only changed elements are updated in real DOM.
- Improves application performance.

What is a Component?

A **component** is a reusable piece of UI.

Example:

- Header
- Footer
- Student card

Types of Components

1. Class Component

```
class App extends Component {  
  render() {  
    return <h1>Hello</h1>;  
  }  
}
```

2. Functional Component

```
function App() {  
  return <h1>Hello</h1>;  
}
```

React Components

- Components are reusable UI blocks.
- Two types: Functional Components and Class Components.
- Helps organize UI into smaller reusable parts.

Understanding JSX

- JSX stands for JavaScript XML.
- Allows writing HTML inside JavaScript.
- Makes UI code easy and readable.

JSX = JavaScript + HTML

```
return <h1>Hello</h1>;
```

Rules:

- Only **one parent element**
- Looks like HTML but is actually JavaScript

JSX Rules

- Must return a single parent element.
- Use className instead of class.
- Tags must be closed.
- Use {} for JavaScript expressions.

Class Components

- Class components are ES6 classes.ES6 is a modern version of JavaScript that introduced new features to make coding easier and more powerful
- Must extend `React.Component`.
- Use `render()` method to display UI.

Props in React

- Props are used to pass data from parent to child component.
- Props are read-only.
- Used for component communication.

State in React

- State stores dynamic data in a component.
- State can be updated.
- UI re-renders when state changes.

State vs Props

- Props are read-only.
- State is mutable.
- Props are passed from parent.
- State is managed within component.

Project Setup

Using:

- Node.js
- npm

Command:

```
npx create-react-app class-demo
```

```
npm start
```

This creates a ready React project

Folder Structure

```
src/  
├─ App.js      (Main component)  
├─ Student.js (Custom component)  
└─ index.js    (Entry point)
```

Entry Point (index.js)

```
import React from "react";  
import ReactDOM from "react-dom";  
import App from "./App";
```

```
ReactDOM.render(<App />, document.getElementById("root"));
```

This loads App into browser

React Render HTML

React's goal is in many ways to render HTML in a web page.

React renders HTML to the web page via a container, and a function called `createRoot()`.

The Container

React uses a container to render HTML in a web page.

Typically, this container is a `<div id="root"></div>` element in the `index.html` file.

CreateRoot & render Functions

The **createRoot** function is located in the **index.js** file in the **src** folder, and is a built-in function that is used to create a root node for a React application.

The render Method

The **render** method defines what to render in the HTML container.

The result is displayed in the **<div id="root">** element.

Coding JSX

JSX stands for JavaScript XML.

JSX allows us to write HTML in React.

JSX makes it easier to write and add HTML in React.

JSX allows us to write HTML elements in JavaScript and place them in the DOM without any `createElement()` and/or `appendChild()` methods.

JSX converts HTML tags into react elements.

```
import React from 'react';
```

```
import ReactDOM from 'react-dom/client';
```

```
import App from './App';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
const myElement = <h1>I Love JSX!</h1>;
```

```
root.render(  
  myElement
```

```
);
```

Create a list with three list items:

index.jsx

```
const myElement = (
```

```
  <ul>
```

```
    <li>Apples</li>
```

```
    <li>Bananas</li>
```

```
    <li>Cherries</li>
```

```
  </ul>
```

```
);
```

JSX

```
import { createRoot } from 'react-dom/client'
const myelement = (
  <table>
    <tr>
      <th>Name</th>
    </tr>
    <tr>
      <td>John</td>
    </tr>
    <tr>
      <td>Elsa</td>
    </tr>
  </table>
);
App.js
createRoot(document.getElementById('root')).render(
  myelement
)
```

Class Component

```
class App extends Component {  
  render() {  
    return <h1>Hello</h1>;  
  }  
}
```

class → ES6 class

extends Component → inherit React features

render() → compulsory method

return → what to display

Without return, nothing is shown

Props (Data Passing)

Props = read-only data passed from parent to child

Example:

App.js

```
<Student name="Saloni" course="MCA" />
```

Student.js

```
<h2>{this.props.name}</h2>
```

```
<p>{this.props.course}</p>
```

Props are **immutable**

Access using `this.props`

What is an Arrow Function?

An **arrow function** is a shorter way to write a function in JavaScript.

Normal Function

```
function add(a, b) {  
  return a + b;  
}
```

Arrow Function

```
const add = (a, b) => {  
  return a + b;  
};
```

Event Handling in React

- React handles events like clicks and form input.
- Uses camelCase syntax.
- Example: `onClick`, `onChange`.
-

Example: Button Click Event

App.js

```
import React from "react";
function App() {
  const handleClick = () => {
    alert("Button Clicked!");
  };
  return (
    <div>
      <h1>Event Handling Demo</h1>
      <button onClick={handleClick}>Click Me</button>
    </div>
  );
}

export default App;
```


Another Example

```
import React, { Component } from "react";  
class App extends Component {  
  constructor() {  
    super();  
    this.state = {  
      count: 0  
    };  
  }  
  increment = () => {  
    this.setState({  
      count: this.state.count + 1  
    });  
  }  
}
```

Example-contd.

```
render() {  
  return (  
    <div>  
      <h1>Class Component Event Handling</h1>  
      <h2>Counter: {this.state.count}</h2>  
      <button onClick={this.increment}>  
        Click Me  
      </button>  
    </div>  
  );  
}  
}  
export default App;
```

Conditional Rendering

- Display UI based on conditions.
- Uses if statements, ternary operator, and && operator.

Conditional Rendering Example

```
import React from "react";
function App() {
  let isLoggedIn = true;
  let message;
  if (isLoggedIn) {
    message = <h2>Welcome User!</h2>;
  } else {
    message = <h2>Please Login</h2>;
  }
  return (
    <div>
      <h1>Conditional Rendering (if statement)</h1>
      {message}
    </div>
  );
}
export default App;
```

Conditional Rendering -Ternary operator

```
import React from "react";
function App() {
  let isLoggedIn = false;
  return (
    <div>
      <h1>Conditional Rendering Example</h1>
      {isLoggedIn ? (
        <h2>Welcome User!</h2>
      ) : (
        <h2>Please Login</h2>
      )}
    </div>
  );
}
export default App;
```

Lists in React

- A list is a collection of items (data)
- Usually stored in an **array**
- Each item needs a unique key.
- Useful for displaying dynamic data.
- React uses arrays and the `map()` function to dynamically display lists of data.
-

App.js

```
import React from "react";
function App() {
  const students = ["Saloni", "Rahul", "Amit", "Neha"];
  return (
    <div>
      <h1>Student List</h1>
      <ul>
        {students.map((name, index) => (
          <li key={index}>{name}</li>
        ))}
      </ul>
    </div>
  );
}
export default App;
```

- The **map() function** is used to loop through the array:

```
students.map((name, index) => ...)
```

- For each student:
 - A `<li>` (list item) is created
 - `name` → displays student name
 - `index` → used as unique **key**
- **key attribute** helps React identify elements efficiently

Forms in React

- Forms are used to **collect user input**
- Common elements:
 - Input fields
 - Textarea
 - Dropdown (select)
 - Buttons

Examples: Login form, Registration form

Form data

- Form data can be accessed using **event handling**
- Data is taken from the **DOM (input fields)**
- Example:
 - `event.target.value`

This approach is called:

◆ **Uncontrolled Component**

- React does NOT directly control input values
- Browser (DOM) manages the data

Event Handling in Forms

- `onChange` → captures input value
- `onSubmit` → processes form
- `event.target.value` → gets user input
- `event.preventDefault()` → stops page reload

Form Example

```
import React from "react";
function App() {
  const handleSubmit = (event) => {
    event.preventDefault();
    const name = event.target.name.value;
    alert("Submitted Name: " + name);
  };
  return (
    <div>
      <h1>Form Example</h1>
      <form onSubmit={handleSubmit}>
        <input type="text" name="name" />

        <button type="submit">Submit</button>
      </form>
    </div>
  );
}
export default App;
```

```
const handleSubmit = (event) => { ... }
```

- This function runs when the form is submitted

```
event.preventDefault();
```

- Stops the page from refreshing after form submission

```
const name = event.target.name.value;- gets the value  
typed by the user
```

- `event.target` → refers to the form
- `name` → input field name
- `value` → entered data

What are Hooks?

Hooks are special functions in **React** that let us use state and other features in functional components.

- Hooks allow functional components to use state and lifecycle features.
- Common hooks: `useState` and `useEffect`.
- Earlier → only class components had state
- Now → functional components can also use state

useState

useState is a Hook used to store and update data in a functional component.

Syntax

```
const [state, setState] = useState(initialValue);
```

Part	Meaning
state	current value
setState	function to update value
initialValue	starting value

useState Example

```
import React, { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}

export default App;
```

Click button → setCount() → state updates → UI updates

useState

- Stores data
- Updates UI automatically
- Replaces class component state

Controlled Components

- In React, form data is handled using **state**
- Input values are controlled by React
- UI and data stay in sync

This is called a **Controlled Component**

Important Attributes

- `value` → binds input to state
- `onChange` → updates state when user types
- `onSubmit` → handles form submission

Data Flow

- User enters data
- `onChange` event triggers
- State gets updated
- UI updates automatically

useState Hook

- Used to manage state in functional components.
- Syntax: `const [state, setState] = useState()`
- Simplifies state management.

Basic Form [Example-App.js](#)

```
import React, { useState } from "react";
function App() {
  const [name, setName] = useState("");
  const handleChange = (event) => {
    setName(event.target.value);
  };
  const handleSubmit = (event) => {
    event.preventDefault();
    alert("Submitted Name: " + name);
  };
  return (
    <div>
      <h1>Form Example</h1>
```

```
<form onSubmit={handleSubmit}>  
  <input  
    type="text"  
    value={name}  
    onChange={handleChange}  
  />
```

```
  <button type="submit">Submit</button>  
</form>
```

```
  <p>Entered Name: {name}</p>  
</div>
```

```
);  
}  
export default App;
```

Explanation

1. Import and useState-useState is used to store input value. It helps React remember data

2. Creating State-const [name, setName] = useState("");

- name → stores entered value
- setName → updates value

3. Handling Input (onChange)-onChange={handleChange}

- Runs when user types
- Captures input value

event.target.value -Gets typed text

4. Updating State- setName(event.target.value);

- Stores user input in state
- Updates UI automatically

5. Form Submission-onSubmit={handleSubmit}-Runs when form is submitted

event.preventDefault();-Stops page from refreshing-
{name}Shows entered value on screen

useEffect Hook

- Used for side effects in components.
- Runs after rendering.
- Used for API calls, DOM updates, timers.

useEffect is used to perform actions when component loads.

In this program:

- It calls backend API automatically
- Fetches student data from Spring Boot
-

Basic Syntax of useEffect

```
useEffect(() => {  
  
}, []);
```

[]- Empty dependency array means:

- run only once when page loads

What is fetch()?

`fetch()` is used to call backend APIs.

```
fetch("http://localhost:8080/McaStudents")
```

Sends request to Spring Boot server

Basic React app to consume your APIs (GET + POST)

Base URL:

<http://localhost:8080/McaStudents>

Method	URL	Purpose
GET	/McaStudents	Get all student
GET	/McaStudents/{id}	Get student by
POST	/McaStudents	Add student

React: Fetch Data from Backend

```
import React, { useEffect, useState } from "react";  
function App() {  
  const [students, setStudents] = useState([]);  
  const [id, setId] = useState("");  
  const [name, setName] = useState("");  
  // GET API  
  useEffect(() => {  
    fetch("http://localhost:8080/McaStudents")  
      .then(res => res.json())  
      .then(data => setStudents(data));  
  }, []);
```

```
// POST API
const addStudent = () => {
  fetch("http://localhost:8080/McaStudents", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      id: id,
      name: name
    })
  })
  .then(res => res.text())
  .then(msg => {
    alert(msg);
    window.location.reload();
  });
};
```

```
return (  
  <div>  
    <h1>MCA Students</h1>  
    { /* ID Input */ }  
    <input  
      type="number"  
      placeholder="Enter ID"  
      value={id}  
      onChange={ (e) => setId(e.target.value) }  
    />  
    <br /><br />  
    { /* Name Input */ }  
    <input  
      type="text"  
      placeholder="Enter Name"  
      value={name}  
      onChange={ (e) => setName(e.target.value) }  
    />  
  </div>  
)
```

```
<br /><br />
  <button onClick={addStudent}>
    Add Student
  </button>
<hr />
{/* Display Students */}
<ul>
  {students.map((s) => (
    <li key={s.id}>
      {s.id} - {s.name}
    </li>
  ))}
</ul>
</div>
);
}
export default App;
```

useEffect

- Runs when component loads
- Used for API calls

useState

- Stores data from backend

fetch()

- Sends HTTP request

◆ **JSON handling**

`response.json()`

What is CORS?

CORS (Cross-Origin Resource Sharing) is a security feature used by browsers.

It controls how a frontend application can access a backend running on a different origin.

An origin = **protocol + domain + port**

Example:

Application

Origin

React app

`http://localhost:3000`

Spring Boot app

`http://localhost:8080`

These are different origins → CORS is triggered

How to Fix CORS in Spring Boot

```
@CrossOrigin(origins = "http://localhost:3000")  
@RestController  
@RequestMapping("/McaStudents")  
@CrossOrigin(origins = "http://localhost:3000")  
public class McaStudentController {
```

Summary

- ReactJS basics
- JSX
- Components
- State and Props
- Event Handling
- Conditional Rendering
- Lists and Forms
- Hooks